Twenty graded practice problems

Difficulty: P01-P05 beginner, P06-P10 developing, P11-P15 intermediate beginner, P16-P20 integrative. Each is worth 5 marks: contract and correct output 2, boundary handling 1, readable explanation 1, own test case 1. Total 100 practice marks, not an invented institutional capstone rubric. Respect declared input types; these are not general-purpose hardened APIs.

Attempt before opening src/solutions.py. Function signatures and runnable solutions appear there and in the notebook.

P01: Greeting

Implement `greeting`. Accept a string name; return Hello, NAME! Preserve supplied whitespace.

Example: `greeting('Asha') -> 'Hello, Asha!'`

Reference solution: `src/solutions.py`, function `greeting`.

P02: Rectangle area

Implement `rectangle_area`. Accept numeric non-negative dimensions; reject negative dimensions with ValueError.

Example: `rectangle_area(3, 4) -> 12`

Reference solution: `src/solutions.py`, function `rectangle_area`.

P03: Temperature

Implement `celsius_to_fahrenheit`. Convert a numeric Celsius temperature using C * 9 / 5 + 32.

Example: `celsius_to_fahrenheit(0) -> 32.0`

Reference solution: `src/solutions.py`, function `celsius_to_fahrenheit`.

P04: Next birthday

Implement `age_next_year`. Accept a non-negative integer age; reject negative age.

Example: `age_next_year(18) -> 19`

Reference solution: `src/solutions.py`, function `age_next_year`.

P05: Character count

Implement `string_length`. Accept a string; return its Python len, including spaces.

Example: `string_length('hi!') -> 3`

Reference solution: `src/solutions.py`, function `string_length`.

P06: Parity

Implement `is_even`. Accept an integer; return a boolean indicating evenness.

Example: `is_even(-2) -> True`

Reference solution: `src/solutions.py`, function `is_even`.

P07: Largest of three

Implement `maximum_of_three`. Accept three comparable numbers; return the largest; allow ties.

Example: `maximum_of_three(3, 3, 1) -> 3`

Reference solution: `src/solutions.py`, function `maximum_of_three`.

P08: Grade bands

Implement `grade`. Accept a score 0..100 inclusive: A >=80, B >=60, C >=40, otherwise F; invalid range raises ValueError.

Example: `grade(80) -> 'A'`

Reference solution: `src/solutions.py`, function `grade`.

P09: Running sum

Implement `sum_to_n`. Accept a non-negative integer; use a loop to sum 1..n inclusive; negative raises ValueError.

Example: `sum_to_n(4) -> 10`

Reference solution: `src/solutions.py`, function `sum_to_n`.

P10: Factorial

Implement `factorial`. Accept a non-negative integer; use a loop; define 0! as 1; negative raises ValueError.

Example: `factorial(4) -> 24`

Reference solution: `src/solutions.py`, function `factorial`.

P11: Vowel count

Implement `count_vowels`. Count ASCII vowels a,e,i,o,u case-insensitively in a string.

Example: `count_vowels('HELLO') -> 2`

Reference solution: `src/solutions.py`, function `count_vowels`.

P12: Reverse string

Implement `reverse_text`. Reverse a string using Python slicing; no special grapheme handling.

Example: `reverse_text('abc') -> 'cba'`

Reference solution: `src/solutions.py`, function `reverse_text`.

P13: List total

Implement `list_total`. Accept a list of numbers; total them with a loop; empty returns 0.

Example: `list_total([2, -1]) -> 1`

Reference solution: `src/solutions.py`, function `list_total`.

P14: Maximum list value

Implement `list_maximum`. Accept a nonempty list of comparable numbers; empty raises ValueError.

Example: `list_maximum([-4, -2]) -> -2`

Reference solution: `src/solutions.py`, function `list_maximum`.

P15: Word counts

Implement `word_frequency`. Lowercase and split a string on whitespace; preserve punctuation in each token.

Example: `word_frequency('Hi hi') -> {'hi': 2}`

Reference solution: `src/solutions.py`, function `word_frequency`.

P16: Normalized palindrome

Implement `is_palindrome`. Ignore non-alphanumeric characters and case using isalnum/lower; empty normalized string counts as a palindrome.

Example: `is_palindrome('Never odd or even') -> True`

Reference solution: `src/solutions.py`, function `is_palindrome`.

P17: Ordered unique values

Implement `unique_in_order`. Accept a list; remove duplicates using equality; preserve first occurrence and do not mutate input.

Example: `unique_in_order([2, 1, 2]) -> [2, 1]`

Reference solution: `src/solutions.py`, function `unique_in_order`.

P18: Safe conversion

Implement `safe_integer`. Accept a string; return int(text), or None if conversion raises ValueError. Do not use eval.

Example: `safe_integer('oops') -> None`

Reference solution: `src/solutions.py`, function `safe_integer`.

P19: Score summary

Implement `score_summary`. Accept a nonempty list of numbers in 0..100; return count, average and highest; invalid range or empty raises ValueError.

Example: `score_summary([50, 100]) -> {'count': 2, 'average': 75.0, 'highest': 100}`

Reference solution: `src/solutions.py`, function `score_summary`.

P20: Longest streak

Implement `study_streak`. Accept a list of booleans; return the longest consecutive run of True; empty returns 0.

Example: `study_streak([True, True, False, True]) -> 2`

Reference solution: `src/solutions.py`, function `study_streak`.